Welcome

Digital and Computer Vision

Lesson 5: Session 1

Final Project – Developing a Real Industry Computer Vision Application

Summary of the **previous** session:Face Recognition with OpenCV & Deep Learning

In the previous session we have covered :

Face Recognition with OpenCV & Deep Learning
Understanding FaceNet and Dlib face recognition models.
Real-time face recognition using OpenCV

Summary of the current session: Final Project – Developing a Real Industry Computer Vision Application

In this session we will covered the following:

Choosing a project.
Data collection, processing, and augmentation.
Training a model using OpenCV and Deep Learning.

Exercise from previous session 1/4

Propose a code to run face recognition from a webcam using keras-facenet for multiple people. Compare the performance of FaceNet and Dlib

```
import os
import cv2
import numpy as np
from keras_facenet import FaceNet
import face_recognition

# Initialize embedder
embedder = FaceNet()

# Load known faces from folders
known_encodings = []
known_names = []

base_dir = "..\\Images\Faces\\"

print("[INFO] Loading known faces...")
```

Exercise from previous session 2/4

```
for person_name in os.listdir(base_dir):
    person_dir = os.path.join(base_dir, person_name)
    if not os.path.isdir(person_dir):
        continue
    for img_name in os.listdir(person_dir):
        if img_name.lower().endswith((".jpg", ".jpeg", ".png")):
            path = os.path.join(person_dir, img_name)
     img = cv2.imread(path)
            if img is None:
                continue
            img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
            face_locations = face_recognition.face_locations(img_rgb)
            if face_locations:
                top, right, bottom, left = face_locations[0]
                face_crop = img_rgb[top:bottom, left:right]
                encoding = embedder.embeddings([face_crop])[0]
                known_encodings.append(encoding)
                known_names.append(person_name)
                print(f"Encoded: {person_name} from {img_name}")
```

Exercise from previous session 3/4

```
        else:
            print(f"[WARNING] No face found in {img_name}")
print("[INFO] Done loading faces.")

# Webcam
cap = cv2.VideoCapture(0)
while True:
    ret, frame = cap.read()
    if not ret:
        break
    rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    face_locations = face_recognition.face_locations(rgb_frame)
    if face_locations:
        faces = [rgb_frame[top:bottom, left:right] for top, right, bottom, left in face_locations]
        embeddings = embedder.embeddings(faces)

        for (top, right, bottom, left), face_embedding in zip(face_locations, embeddings):
            distances = np.linalg.norm(np.array(known_encodings) - face_embedding, axis=1)
            min_distance = np.min(distances)
            best_match_index = np.argmin(distances)
```

Exercise from previous session 4/4

```
            name = "Unknown"
            if min_distance < 0.8:
                name = known_names[best_match_index]

            # Draw bounding box and label
            cv2.rectangle(frame, (left, top), (right, bottom), (0, 255, 0), 2)
            cv2.putText(frame, name, (left, top - 10),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.75, (0, 255, 0), 2)

    cv2.imshow("FaceNet Face Recognition", frame)
    if cv2.waitKey(1) & 0xFF == ord("q"):
        break

cap.release()
cv2.destroyAllWindows()
```

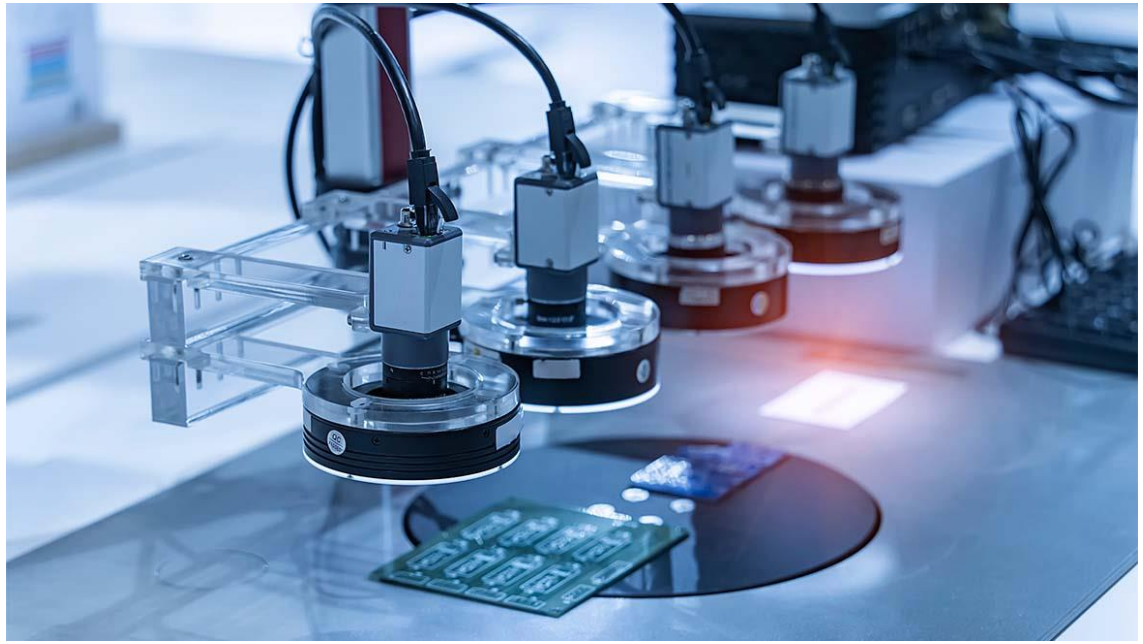
Final project ideas for a Computer Vision with Python, OpenCV, and Deep Learning

We are proposing 9 strong final project ideas for our course using Python, OpenCV, and Deep Learning — all designed to match real-world industry applications across different sectors.

You have to describe the algorithm, find data for training and testing, develop the code and test the application. Finally you have to discuss the result : speed performance, precision and robustness.

1. Industrial Quality Control System

Goal: Detect surface defects (cracks, dents, missing parts) on manufactured components (e.g., metal or plastic parts) using image classification and object detection. This is one of the most practical and widely used computer vision applications in real manufacturing environments.



1. Industrial Quality Control System - Preprocessing

Ideas:

Convert to grayscale or use specific color channels (e.g., HSV for contrast).

Apply Gaussian blur or bilateral filtering to remove noise.

Use adaptive thresholding, edge detection (Canny), or morphological operations to highlight defects.

1. Industrial Quality Control System – CNN Deep Learning classification

Ideas:

Use CNNs for classification:

Train a binary classifier: Defective vs. Non-defective

Or a multi-class classifier if you have defect types (scratch, crack, hole, etc.)

Use architectures like:

MobileNetV2 or EfficientNet for speed

ResNet or VGG for accuracy

You can also use object detection (YOLO or EfficientDet) to localize defects.

1. Industrial Quality Control System – Public Datasets for Industrial Defect Detection

1. NEU Surface Defect Database
Steel surface defect images
Categories: scratch, rolled-in scale, patches, crazing, pitted surface

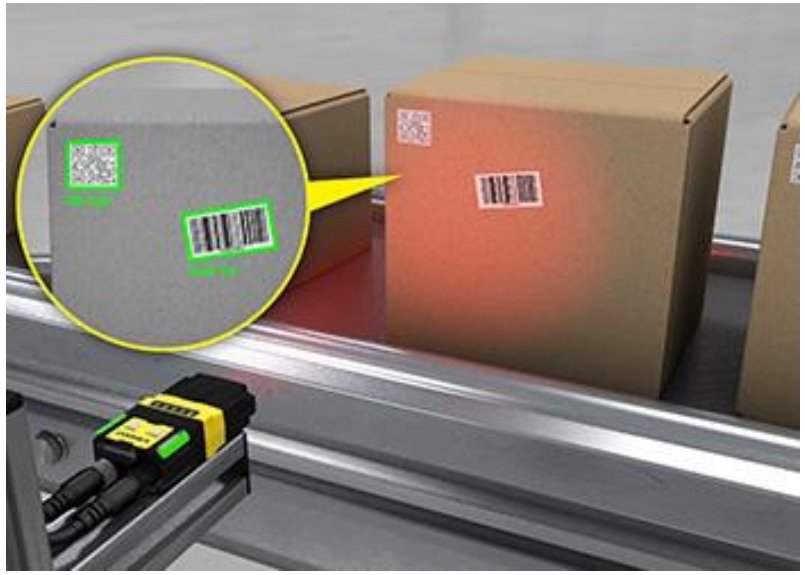
2. DAGM Dataset
Textured surfaces with artificial defects
Useful for segmentation/classification

3. Severstal Steel Defect Dataset (Kaggle)
Pixel-level defect annotations
Great for segmentation models like U-Net

4. MVTec AD Dataset
Real industrial data for anomaly detection
Categories: Metal nut, transistor, tile, wood, leather, etc.

2. Automated Package Dimensioning and Barcode Reader

Goal: Measure dimensions of packages and detect barcodes from images/videos for warehouse automation. Often used in logistics, warehousing, and shipping industries (e.g., Amazon, DHL, UPS).



2. Automated Package Dimensioning and Barcode Reader - Measuring the Package Size

Input:
A webcam or smartphone camera capturing a top-down or angled view of the package on a background with a reference object like a known-sized square or marker.

Core Idea
Use image processing and perspective transformation to compute real-world dimensions.

Steps:
Preprocess the image (grayscale, blur, threshold)
Detect contours and isolate the box
Use a reference object (e.g., A4 paper, a coin, ArUco marker) to compute the scale (pixels per metric)
Calculate dimensions using Euclidean distances between points

2. Automated Package Dimensioning and Barcode Reader - Barcode or QR Code Reader

Libraries:
pyzbar: Reads barcodes & QR codes
opencv-contrib-python: for ArUco marker-based measurement
zxing or opencv QR detector (for alternative methods)

Public Datasets for Testing:
Barcode dataset on Roboflow
QR Codes: Generate your own using online tools or Python qrcode package.

3. Smart Surveillance with Real-Time Intrusion Detection & Face Recognition

Goal: Create an intelligent surveillance system that:
Detects people entering a secure area in real-time.
Recognizes known individuals using face recognition.
Raises an alert (sound/email/log) if an unknown person (intruder) is detected.

System Pipeline:
1. Capture: Get video feed from webcam or IP camera
2. Motion detection: Trigger next stages only motion is detected
2. Detect Person: Use real-time object detector to filter only people
3. Detect Face: Extract faces from detected persons
4. Recognize Face: Match against known faces in a database
5. Trigger Alert: If face not recognized → log, sound alarm, or notify admin

3. Smart Surveillance with Real-Time Intrusion Detection & Face Recognition - Data Sources

You can use your own images for face recognition

For intrusion datasets: simulate scenes or use public datasets like:
AI City Challenge
Open Images Dataset
COCO dataset for person bounding boxes

3. Smart Surveillance with Real-Time Intrusion Detection & Face Recognition - Real-World Applications

Smart home security

Restricted access zones

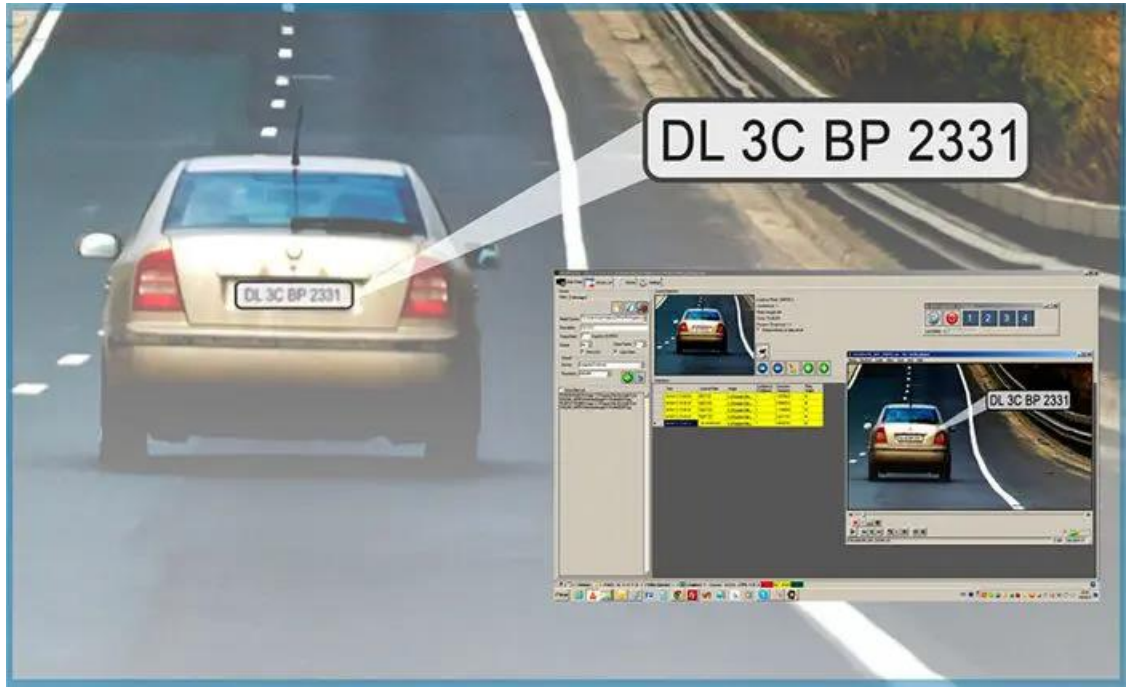
School/university attendance via face

Warehouse monitoring

Factory floor safety zones

4. Automatic Vehicle License Plate Recognition (ALPR)

Goal: Detect cars and recognize license plates from street or parking lot footage. Automatic Vehicle License Plate Recognition (ALPR) is a real-world, high-impact computer vision application widely used in security, traffic monitoring, parking management, toll collection, and law enforcement.



4. Automatic Vehicle License Plate Recognition (ALPR) - Pipeline Overview

ALPR is a multi-stage computer vision pipeline that:

1. Detects a vehicle

2. Finds and crops the license plate region

3. Segments the characters.

4. Recognizes characters using OCR (Optical Character Recognition)

4. Automatic Vehicle License Plate Recognition (ALPR) - Technologies, Tools and Databases

Plate Detection: OpenCV (Haar cascades), YOLO, SSD, or EasyOCR
Preprocessing: OpenCV (resize, denoise, thresholding, morph)
OCR: Tesseract OCR, EasyOCR
Full-stack ALPR: OpenALPR (commercial and open-source), HyperLPR (Chinese plates), PaddleOCR

Public Datasets for Training or Testing:
OpenALPR Benchmark Dataset
CCPD Dataset (Chinese plates)
UFPR-ALPR Dataset
License Plate Dataset by Kaggle

5. Smart Retail: Customer Analytics and Product Detection

Goal: Track customers in a store, detect picked-up products, and generate heatmaps of store activity. Predict out-of-stock shelves or busiest times. This project is a perfect blend of computer vision, deep learning, and business intelligence. It tackles real-world retail challenges like shelf management, customer behavior tracking, and inventory optimization.

5. Smart Retail: Customer Analytics and Product Detection - Key Modules

1. Product Detection on Shelves
Detect and count products in specific areas (e.g., shelf rows).
Alert when product count drops below a threshold.
Helps predict out-of-stock situations.

2. Customer Detection & Movement Tracking
Track how many customers enter a zone (aisle, shelf).
Map time spent at specific shelves.
Track movement over time (trajectory, heatmap).

3. Predict Out-of-Stock Scenarios
Use object detection + time-series models (e.g., LSTM) to forecast depletion.
Optionally integrate with sales data or inventory databases.

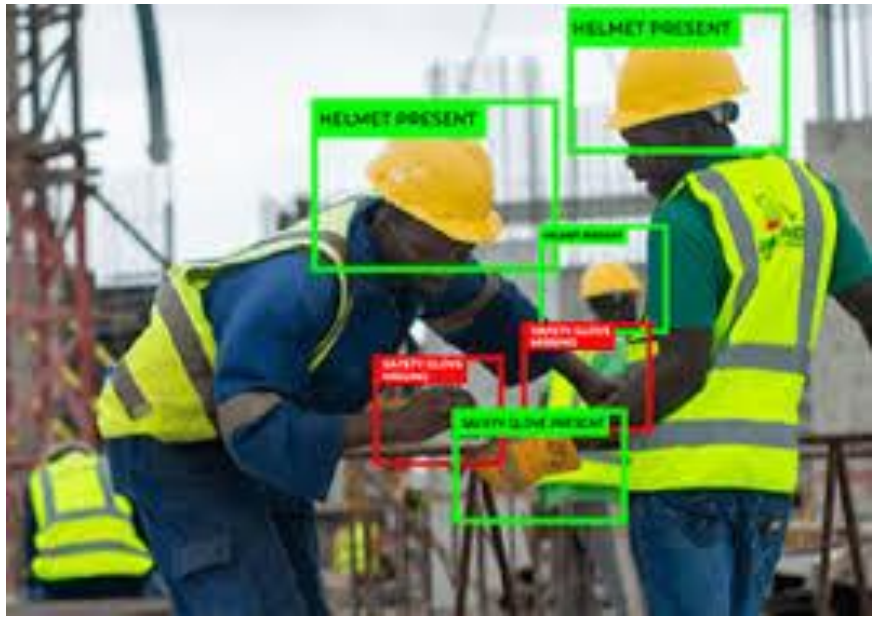
4. Busiest Hours Analytics
Aggregate hourly customer counts to find peak shopping times.
Useful for staffing or targeted promotions.

5. Smart Retail: Customer Analytics and Product Detection - Public Datasets for Training / Testing

SKU110K	11k images of store shelves, great for product detection
D2S Dataset	Annotated supermarket items
Mall Dataset	Foot traffic and crowd analytics in a shopping mall
AI City Challenge Dataset	Includes tracking and retail analytics scenarios

6. PPE (Helmet/Mask/Gloves) Detection in Construction Sites

Goal: Real-time monitoring system to ensure workers wear safety gear. This is a practical safety-critical computer vision application, increasingly used in smart surveillance and industrial compliance monitoring. It's a fantastic real-world project that demonstrates object detection, classification, and even real-time alerting systems.



6. PPE (Helmet/Mask/Gloves) Detection in Construction Sites- Components of the System

1. PPE Object Detection.
Helmet
Face mask
Gloves
Vest (optional)
Person

2. Match detected gear with detected person → Determine compliance

3. Violation Detection
If a person is missing any of the required PPE (e.g., no helmet), flag them.
Draw bounding boxes in red for violations, green for compliance.

4. Alert System
Beep or alert on screen
Save violation frames or logs
Optional real-time integration (e.g., MQTT, SMS, email)

6. PPE (Helmet/Mask/Gloves) Detection in Construction Sites- Tools and Models

1. Object Detection Models:
YOLOv5 / YOLOv8: Highly accurate, fast, real-time object detector
EfficientDet: Good accuracy, mobile-friendly
SSD MobileNet: Lightweight, good for edge deployment

2. Face Mask Detection:
Fine-tuned MobileNet or ResNet classifier for face mask detection

6. PPE (Helmet/Mask/Gloves) Detection in Construction Sites - Public Datasets for PPE Detection

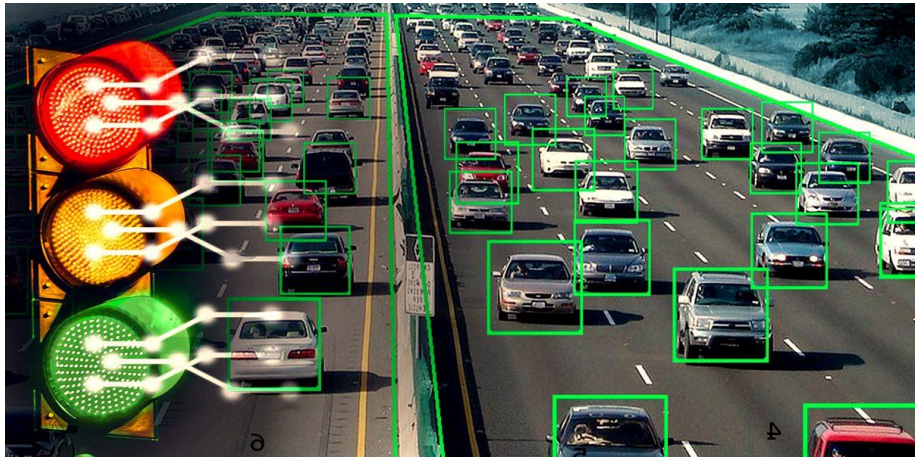
PPE Detection Dataset (Roboflow) Includes person, helmet, vest
Safety-PPE-Detection Custom-trained PPE model using ImageAI
Kaggle Construction PPE Dataset Annotated images of construction workers wearing/not wearing helmets
Face Mask Detection Dataset For mask detection

7. Traffic Sign and Violation Detection

Goal: Detect traffic signs and recognize vehicle violations like red-light jumping or wrong-way driving. This is a powerful and real-world computer vision application, especially in the context of smart cities, intelligent transportation systems (ITS), and automated traffic law enforcement. It involves detecting traffic signs, monitoring vehicle behavior, and identifying violations like red-light running, speeding, or wrong-way driving.

Develop a system that can:

1. Detect and recognize traffic signs (stop, speed limit, no entry, etc.)
2. Track vehicle movements and monitor adherence to rules
3. Detect violations like:
   - Ignoring stop signs
   - Speeding
   - Red light jumping
   - Wrong-way driving
   - Lane violations
4. Log incidents for reporting or analysis



7. Traffic Sign and Violation Detection - Tools and Models

Traffic Sign Detection	YOLOv5, YOLOv8, EfficientDet	PyTorch, OpenCV
Traffic Sign Classification	CNN, MobileNet, ResNet	TensorFlow/Keras
Vehicle Detection	YOLOv5, OpenCV DNN, SSD	OpenCV, PyTorch
Object Tracking	SORT / Deep SORT / ByteTrack	sort, strongsort, opencv-contrib-python
Violation Logic	Custom code (Python)	OpenCV, Pandas
Logging & Reporting	Custom code (Python) save violation data: Vehicle ID / track ID Timestamp Type of violation Cropped image or bounding box	

7. Traffic Sign and Violation Detection - Datasets

German Traffic Sign Recognition Benchmark (GTSRB)

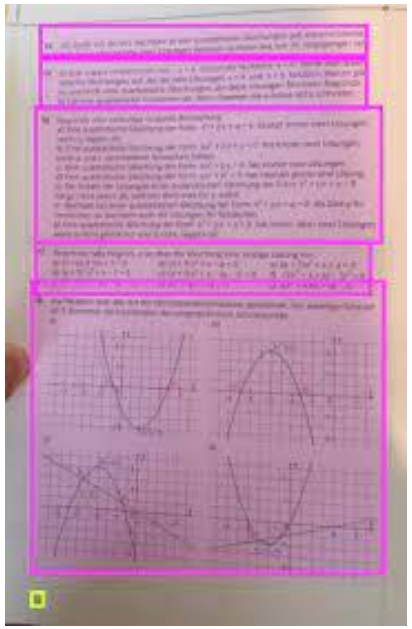
BelgiumTS

Mapillary Traffic Sign Dataset

LISA Traffic Sign Dataset

8. Document Scanner and Text Extractor for Digitization

Goal: Build a tool that scans, flattens, and OCRs printed documents into editable text. This is a practical and highly impactful application of computer vision and OCR (Optical Character Recognition), especially relevant for offices, archives, libraries, banks, and education sectors aiming to digitize paper documents.



8. Document Scanner and Text Extractor for Digitization

Goal: Build a tool that scans, flattens, and OCRs printed documents into editable text. This is a practical and highly impactful application of computer vision and OCR (Optical Character Recognition), especially relevant for offices, archives, libraries, banks, and education sectors aiming to digitize paper documents.

System pipeline:
1. Scans paper documents using a webcam or image input
2. Detects document edges, applies perspective transformation (as if scanned)
3. Enhances the image (binarization, denoising, sharpening)
4. Extracts text via OCR (Optical Character Recognition)
5. Saves results as PDF, searchable text, or metadata

8. Document Scanner and Text Extractor for Digitization - Tools & Libraries

Document detection	OpenCV (contours, edges)
Perspective transform	OpenCV
Image pre-processing	OpenCV
OCR	Tesseract OCR, EasyOCR
Export to PDF/Text	reportlab, fpdf, pdf2image

8. Document Scanner and Text Extractor for Digitization - Public Datasets (For Training / Testing)

DocBank – Layout-aware document dataset

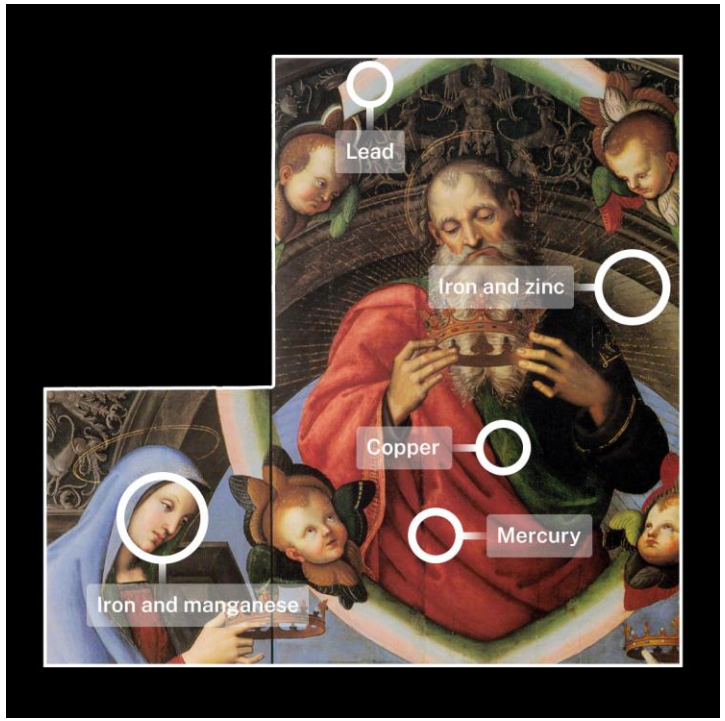
RVL-CDIP – 400,000 labeled scanned pages

CORD – Receipts and structured text

FUNSD – Forms Understanding in Noisy Scanned Documents

9. AI Art Inspector – Forgery or Damage Detection

Goal: Analyze paintings or prints to detect authenticity, cracks, color degradation, or tampering. This project combines computer vision, deep learning, and even a little digital forensics to analyze artworks for signs of forgery, alterations, or physical damage. It's a sophisticated and exciting application with real-world relevance in museums, auction houses, insurance, and conservation labs.



9. AI Art Inspector – Forgery or Damage Detection - Core Computer Vision Tasks

Style consistency	CNN-based feature extraction (style embeddings)
Damage detection	Image segmentation / anomaly detection
Forgery detection	Siamese networks / image comparison
Texture & brushstroke analysis	Gabor filters, wavelets, high-res image features
UV/IR simulation	Simulated filters for revealing underlying layers
Metadata extraction	OCR on artist signatures, labels, annotations

9. AI Art Inspector – Forgery or Damage Detection - Technologies

Deep Learning: Keras, TensorFlow, PyTorch

CV Libraries: OpenCV, scikit-image

Pretrained Models: ResNet, VGG, EfficientNet, DINOv2 for visual similarity

Siamese Networks: To compare original vs suspect artwork

Unsupervised Methods: Autoencoders, GANs for anomaly detection

OCR: For artist name/date label extraction

9. AI Art Inspector – Forgery or Damage Detection - Datasets & Resources

WGA Dataset	Web Gallery of Art – 45k+ classical paintings
WikiArt Dataset	Paintings categorized by style/artist
ForgeryNet (Alibaba DAMO)	Real and forged paintings dataset
Kaggle: Crack Detection in Paintings	For crack and wear classification
Your own high-res scans/photos	From museums or art catalogues

In this session we have covered:

Final Project – Developing a Real Industry Computer Vision Application
Choosing a project.
Data collection, processing, and augmentation.
Training a model using OpenCV and Deep Learning.

In the next session we will covered:

Deployment & Course Recap

THANKS !!!